

1.5 Boolean Expressions

Sam is reading his scriptures one day and comes across the following verse from 2 Corinthians:

Every man according as he purposeth in his heart, so let him give; not grudgingly, or of necessity: for God loveth a Cheerful giver. (2 Corinthians 9:7)

Now he wonders: is his offering acceptable to the Lord? To address this issue, he reduces the scripture to a Boolean expression.

Objectives

By the end of this class, you will be able to:

- Declare a Boolean variable.
- Convert a logic problem into a Boolean expression.
- Recite the order of operations.

Prerequisites

Before reading this section, please make sure you are able to:

- Represent simple equations in C++ (Chapter 1.3).
- Choose the best data-type to represent your data (Chapter 1.2).

Overview

Boolean algebra is a way to express logical statements mathematically. This is important because virtually all programs need to have decision making logic. There are three parts to Boolean algebra: Boolean variables (variables enabling the programmer to store the results of Boolean expressions), Boolean operators (operations that can be performed on Boolean variables), and Comparison operators (allowing the programmer to convert a number to a Boolean value by comparing it to some value). The most common operators are:

Math	English	C++	Example
~	Not	!	!true
^	And	&&	true && false
∨	Or		true false
=	Equals	==	x + 5 == 42 / 2
≠	Not Equals	!=	graduated != true
<	Less than	<	age < 16
≤	Less than or equal to	<=	timeNow <= timeLimit
>	Greater than	>	age > 65
≥	Greater than or equal to	>=	grade >= 90

And, Or, and Not

The three main logical operators we use in computer programming are And, Or, and Not. These, it turns out, are also commonly used in our spoken language as well. For example, consider the following scripture:

Every man according as he purposeth in his heart, so let him give; not grudgingly, or of necessity: for God loveth a Cheerful giver. (2 Corinthians 9:7)

This can be reduced to the following expression:

acceptable = inHisHeart and not (grudgingly or necessity)

In C++, this will be rendered as:

```
bool isAcceptable = isFromHisHeart && !(isGrudgingly || isOfNecessity);
```

This Boolean expression has all three components: And, Or, and Not.

AND

The Boolean operator AND evaluates to true only if the left-side and the right-side are both true. If either are false, the expression evaluates to false. Consider the following statement containing a Boolean AND expression:

```
bool answer = leftSide && rightSide;
```

This can be represented with a truth-table:

AND		Left-side	
		true	false
Right-side	true	true	false
	false	false	false

If leftSide = false and rightSide = false, then leftSide && rightSide evaluates to false. This case is represented in the lower-right corner of the truth table (observe how the column corresponding to that cell has false in the header corresponding to the leftSide variable. Observe how the row corresponding to that cell has false in the header corresponding to the rightSide variable).

The AND operator is picky: it evaluates to true only when both sides are true.

OR

The Boolean operator OR evaluates to `true` if either the left-side or the right-side are `true`. If both are `false`, the expression evaluates to `false`. Consider the following statement containing a Boolean OR expression:

```
bool answer = leftSide || rightSide;
```

The corresponding truth-table is:

OR		Left-side	
		true	false
Right-side	true	true	true
	false	true	false

If `leftSide = false` and `rightSide = true`, then `leftSide || rightSide` evaluates to `true`. This case is represented in the middle-right cell of the truth table (observe how the column corresponding to that cell has `false` in the header corresponding to the `leftSide` variable. Also note how the row corresponding to that cell has `true` in the header corresponding to the `rightSide` variable).

The OR operator is generous: it evaluates to `true` when either condition is met.

NOT

The Boolean operator NOT is a unary operator: only one operand is needed. In other words, it only operates on the value to the right of the operator. NOT evaluates to `true` when the right-side is `false` and evaluates to `false` with the right-side is `true`. Consider the following statement containing a Boolean NOT expression:

```
bool wrong = ! right;
```

The corresponding truth-table is:

NOT	
Right-side	
true	false
false	true

If `right = false` then `!right` is `true`. If `right = true` then `!right` is `false`. In other words, the NOT operator can be thought of as the “opposite operator.”

Example

Back to our scripture from the beginning:

Every man according as he purposeth in his heart, so let him give; not grudgingly, or of necessity: for God loveth a Cheerful giver. (2 Corinthians 9:7)

This, as we discussed, is the same as:

```
bool isAcceptable = isFromHisHeart && !(isGrudgingly || isOfNecessity);
```

In this case, Sam is giving from his heart (`isFromHisHeart = true`) and is not giving of necessity (`isOfNecessity = false`). Unfortunately, he is a bit resentful (`isGrudgingly = true`). Evaluation is:

1. `bool isAcceptable = isFromHisHeart && !(isGrudgingly || isOfNecessity);`
2. `bool isAcceptable = true && !(true || false);` // replace variables with values
3. `bool isAcceptable = true && !(true);` // `true || false --> true`
4. `bool isAcceptable = true && false;` // `!true --> false`
5. `bool isAcceptable = false;` // `true && false --> false`

Thus we can see that Sam's offering is not acceptable to the Lord. The grudging feelings have wiped out all the virtue from his sacrifice.

Sam's Corner



The more transformations you know, the easier it will be to work with Boolean expressions in the future. Consider the distributive property of multiplication over addition:

$$a * (b + c) == (a * b) + (a * c)$$

Knowing this algebraic transformation makes it much easier to solve equations. There is a similar Boolean transformation called DeMorgan. Consider the following equivalence relationships:

$$!(p || q) == !p \&\& !q$$

$$!(p \&\& q) == !p || !q$$

It also works for AND/OR:

$$a || (b \&\& c) == (a || b) \&\& (a || c)$$

$$a \&\& (b || c) == (a \&\& b) || (a \&\& c)$$



Sue's Tips

Boolean operators also work with numbers as well. Recall that `0` $\rightarrow$ `false` and all values other than `0` map to `true`. When evaluating Boolean expressions containing non-Boolean values, you convert the value to a `bool` immediately before the Boolean operator is evaluated:

$$(7 \&\& 0) \rightarrow (true \&\& false) \rightarrow false$$

$$!65 \rightarrow !true \rightarrow false$$

Comparison Operators

Boolean algebra only works with Boolean values, values that evaluate to either `true` or `false`. Often times we need to make logical decisions based on values that are numeric. Comparison operators allow us to make these conversions.

Equivalence

The first class of comparison operators consists of statements of equivalence. There are two such operators: equivalence `==` and inequality `!=`. These operators will determine whether the values are the same or not. Consider the following code:

```
int grade = 100;
bool isPerfectScore = (grade == 100);
```

In this example, the Boolean variable `isPerfectScore` will evaluate to `true` only when `grade` is 100%. If `grade` is any other value (including 101%), `isPerfectScore` will evaluate to `false`. It is also possible to tell if two values are not the same:

```
int numStudents = 21;
bool isClassHeldToday = (numStudents != 0);
```

Here we can see that we should go to class today. As long as the number of students attending class (`numStudents`) does not equal zero, class is held.

Relative Operators

The final class of comparison operators performs relative (not absolute) evaluations. These are greater than `>`, less than `<`, greater than or equal to `>=`, and less than or equal to `<=`. Consider the following example using integers:

```
int numBoys = 6;
int numGirls = 8;
bool isMoreGirls = (numGirls > numBoys);
```

This works in much the same way when we compare floating point numbers. Note that since floating point numbers (`float`, `double`, `long double`) are approximations, there is little difference between `>` and `>=`.

```
float grade = 82.5;
bool hasPassedCS124 = (grade >= 60.0); // passed greater than or equal to 60%
```

Finally, we can even use relative operators with chars. In these cases, it is important to remember that each letter in the ASCII table corresponds to a number. While we need not memorize the ASCII table, it is useful to remember that the letters are alphabetical and that uppercase letters are before lowercase letters:

```
char letterGrade = 'B';
bool goodGrade = ('C' >= letterGrade);
```

Example 1.5 – Decision Function

Demo	This example will demonstrate how to write a function to help make a decision. This will be a binary decision (choosing between two options) so the return type will be a <code>bool</code> .
Problem	Write a program to compute whether a user qualifies for the child tax credit. The rule states you qualify if you make less than \$110,000 a year (the actual rule is quite a bit more complex, of course!). Note that you either qualify or you don't: there are only two possible outcomes. If you do qualify, then the credit is \$1,000 per child. If you don't, no tax credit is awarded. <pre>What is your income: <u>115000.00</u> How many children? 2 Child Tax Credit: \$ 0.00</pre>
Solution	The key part of this problem is the function deciding whether the user qualifies for the child tax credit. The input is income as a <code>float</code> and the output is the decision as a <code>bool</code>. <pre> /***** * QUALIFY * Does the user qualify for the tax credit? * This will return a BOOL because you either * qualify, or you don't! *****/ bool qualify(double income) { return (income <= 110000.00); }</pre> Observe how the name of the function implies what true means. In other words, if <code>qualify()</code> returns true, then the user qualifies. If <code>qualify()</code> returns false, then the user doesn't. Always make sure the name of the function implies what true means when working with a <code>bool</code> function. The next part is computing the credit to be awarded. This will require an IF statement which will be discussed next chapter. <pre> if (qualify(income)) cout << 1000.00 * (float)numChildren << endl; else cout << 0.00 << endl;</pre> Notice how the return value of the <code>qualify()</code> function goes directly into the IF statement.
Challenge	It turns out that the child tax credit is actually more complex than this. The taxpayer gets the full \$1,000 for every child if the income is less than \$110,000 but it phases out at the rate of 5¢ for each \$1 after that. In other words, a family making \$120,000 will only receive \$500 per child. Thus there is no credit possible for families making more than \$130,000. As a challenge, modify the above example to more accurately reflect the law.
See Also	The complete solution is available at 1-5-decisionFunction.cpp or: <pre>/home/cs124/examples/1-5-decisionFunction.cpp</pre> 

Order of Operations

With all these Boolean operators, the order of operations table has become quite complex (a more complete version of this table is in [Appendix B](#)):

()	Parentheses				
++ --	Increment, decrement			Math	Unary
!	Not			Logic	
* / %	Multiply, divide, modulo			Math	Binary
+ -	Addition, subtraction				
> >= < <=	Greater than, less than, etc.		Relative	Logic	
== !=	Equality		Absolute		
&&	And		AND		
	Or		OR		
= += *=	Assignment				

There are a couple things to remember when trying to memorize the order of operations:

1. **Unary Before Binary:** When an operator only takes one operand (such as `x++` or `!true`), it goes at the top of the table. When an operator takes two (such as `3 + 6` or `grade > 60`), it goes at the bottom of the table.
2. **Math Before Logic:** Arithmetic operators (such as addition or multiplication) go before Boolean operators (such as AND or Greater-than). This means that operations evaluating to a `bool` go after operations evaluating to numbers.
3. **Relative Before Absolute:** Conditional operators making a relative comparison (such as greater-than `>`) go before those making absolute comparisons (such as not-equal `!=`).
4. **AND Before OR:** This is one of those things to just memorize. Possibly you can remember that they are in alphabetical order?



Sue's Tips

While it is useful (and indeed necessary!) to memorize the order of operations, please don't expect the readers of your code to do the same. It is far better to disambiguate your expressions by using many parentheses. This gives the bugs nowhere to hide!

Problem 1, 2

Write a function to multiply two numbers. Call the function `multiply()`.

Write `main()` to prompt the user for two numbers, call `multiply()`, and display the product.

Please see page 64 for a hint.

Problem 3

Write a function to represent the prerequisites for CS 165: you must pass CS 124 and Math 110.

Please see page 81 for a hint.

Problem 4

Write a function to represent how to pass this class: you can either earn a grade greater than or equal to 60% or you must bribe the professor. Realize, of course, that this is not how to pass the class...

Please see page 81 for a hint.

Problem 5-11

What is the value for each of the following variables?

{	
bool a = ('a' < 'a');	
bool b = ('b' > 'a');	
bool c = (a * 4) && b;	
bool d = !(b (c true));	
bool e = a && b && c && d;	
bool f = a b c d;	
bool g = (a != b) && true;	
}	

Please see page 82 for a hint.

Problem 12-16

For each of the following, indicate where the parentheses goes to disambiguate the order of operations:

Raw expression	With parentheses
1 + 2 > 3 * 3	
! a < b	
a + b && c d	
2 * c++ > 2 + 7 == 9 % 2	
a > b > c > d	

Please see page 49 for a hint.

Assignment 1.5

Write a function to determine if an individual is a full tithe payer. This program will have one function that accepts as parameters the income and payment, and will return whether or not the user is a full tithe payer. The return type will need to be a Boolean value. Note that `main()` is already written for you. Also note that the skeleton of `isFullTithePayer()` is written, but there is more code to be written in the function for it to work as desired.

For this assignment, `main()` will be provided at:

```
/home/cs124/assignments/assign15.cpp
```

Please copy this file and use it as you did the templates up to this point.

Example

Two examples. The user input is in underline.

Example 1: Full Tithe Payer

```
Income: 100  
Tithe: 91  
You are a full tithe payer.
```

Example 2: Not a Full Tithe Payer

```
Income: 532  
Tithe: 40  
Will a man rob God? Yet ye have robbed me.  
But ye say, Wherein have we robbed thee?  
In tithes and offerings. Malachi 3:8
```

Instructions

The test bed is available at:

```
testBed cs124/assign15 assignment15.cpp
```

Don't forget to submit your assignment with the name "Assignment 15" in the header.

Please see page 81 for a hint.